
Especificación de requisitos de software

Proyecto: Hotel Transilvania
Revisión 01

03/10/2025



Ficha del documento

Fecha	Revisión	Autor	Verificado dep. calidad.
03/10/2025	1.0	Rubinho Martinez, Carlos Gaitan, Santiago Sierra	[Firma o sello]

Documento validado por las partes en fecha: [Fecha]

Por el cliente	Por la empresa suministradora
Fdo. D./ Dña [Nombre]	Fdo. D./Dña [Nombre]

Contenido

FICHA DEL DOCUMENTO	2
CONTENIDO	3
1 INTRODUCCIÓN	5
1.1 Propósito	5
1.2 Alcance	5
1.3 Personal involucrado	5
1.4 Definiciones, acrónimos y abreviaturas	5
1.5 Referencias	5
1.6 Resumen	6
2 DESCRIPCIÓN GENERAL	7
2.1 Perspectiva del producto	7
2.2 Funcionalidad del producto	7
2.3 Características de los usuarios	7
2.4 Restricciones	7
2.5 Suposiciones y dependencias	7
2.6 Evolución previsible del sistema	8
3 REQUISITOS ESPECÍFICOS	8
3.1 Requisitos comunes de los interfaces	9
3.1.1 Interfaces de usuario	¡Error! Marcador no definido.
3.1.2 Interfaces de hardware	9
3.1.3 Interfaces de software	9
3.1.4 Interfaces de comunicación	10
3.2 Requisitos funcionales	10
3.2.1 Requisito funcional 1	10
3.2.2 Requisito funcional 2	¡Error! Marcador no definido.
3.2.3 Requisito funcional 3	¡Error! Marcador no definido.
3.2.4 Requisito funcional n	¡Error! Marcador no definido.
3.3 Requisitos no funcionales	14
3.3.1 Requisitos de rendimiento	14
3.3.2 Seguridad	14
3.3.3 Fiabilidad	14
3.3.4 Disponibilidad	15

3.3.5	Mantenibilidad	15
3.3.6	Portabilidad	15
3.4	Otros requisitos	15
4	APÉNDICES	16

1 Introducción

[Inserte aquí el texto]

La introducción de la Especificación de requisitos de software (SRS) debe proporcionar una vista general de la SRS. Debe incluir el objetivo, el alcance, las definiciones y acrónimos, las referencias, y la vista general del SRS.

1.1 Propósito

El propósito de este documento es especificar, de forma completa y verificable, los requisitos funcionales y no funcionales del Sistema de Gestión de Reservas para un Hotel Transilvania. Está dirigido a: cliente (propietario del hotel), equipo de desarrollo (implementación en Python), equipo de pruebas, departamento de calidad y partes interesadas.

1.2 Alcance

Sistema de Gestión de Reservas para un Hotel.

Funcionalidades principales: gestión de reservas, clientes, habitaciones y pagos.

Implementado en Python aplicando POO, sin interfaz gráfica en esta versión. Entregable en plazo máximo de 15 días (3 sprints de 5 días) con documentación técnica.

1.3 Personal involucrado

Nombre	Santiago Sierra
Rol	Project Manager
Categoría profesional	Scrum Master
Responsabilidades	Planificación global, coordinación con cliente, gestión del cronograma, facilita Scrum y elimina impedimentos
Información de contacto	ludwingsantiagosierraquintana@gmail.com
Aprobación	Aprobado

Nombre	Rubinho Martinez
Rol	Product Owner
Categoría profesional	Analista de Requisitos
Responsabilidades	Planificación global, coordinación con cliente, gestión del cronograma, facilita Scrum y elimina impedimentos
Información de contacto	bautistamarbau7@gmail.com
Aprobación	Aprobado

Nombre	Carlos Gaitan
Rol	Desarrollador líder
Categoría profesional	QA Tester / DevOps de apoyo
Responsabilidades	Codificación (POO, reservas y pagos), integración de módulos, soporte en pruebas y despliegue
Información de contacto	luiscarlosgaitasena@gmail.com
Aprobación	Aprobado

1.4 Definiciones, acrónimos y abreviaturas

- SRS: Software Requirements Specification
- RF: Requisito Funcional
- RNF: Requisito No Funcional
- POO: Programación Orientada a Objetos

- CLI: Command Line Interface (interfaz de consola)
- DB: Base de datos (archivo local en formato JSON/SQLite según implementación)

1.5 Referencias

Referencia	Título	Ruta	Fecha	Autor
1	Plantilla de Especificación de Requisitos de Software (SRS)	IEEE Std 830-1998	1998	IEEE Computer Society
2	Guía de estilo y formato para documentación técnica de software	DOC-TEC/2024-01	Enero 2024	Departamento de Ingeniería de Software
3	Guía de desarrollo ágil con metodología Scrum	SCRUM-GUIDE-2020	Noviembre 2020	Scrum.org
4	Guía de estilo de código Python (PEP 8)	PEP 8 – Python Enhancement Proposal	Actualización 2023	Python Software Foundation
5	Manual de uso de base de datos SQLite 3	SQLT3-USER-GUIDE	Junio 2022	SQLite Consortium
6	Documento de alcance del proyecto: Sistema de Gestión de Reservas para un Hotel	DOC-ALC-HOTEL-01	Agosto 2025	Equipo de Proyecto (Rubinho Martínez, Carlos Gaitán, Santiago Sierra)
7	Plan de trabajo y cronograma de desarrollo (Metodología Scrum)	PLAN-SCRUM-RESERVAS	Agosto 2025	Equipo de Desarrollo DNILRONALD Software
8	Guía de pruebas unitarias y de integración (Pytest Framework)	QA-TEST-PYT/2023	Mayo 2023	QA Team – Open Source Community
9	Ley 1581 de 2012 – Protección de Datos Personales (Colombia)	LEY1581/2012	Octubre 2012	Congreso de la República de Colombia
10	Política interna de manejo y respaldo de datos del Hotel	POL-DAT/2025	Julio 2025	Hotel Cliente – Área de Sistemas

1.6 Resumen

Este documento describe el contexto del sistema, los requisitos de interfaces, los requisitos funcionales (numerados), requisitos no funcionales medibles y apéndices con detalles adicionales (modelo de datos y ejemplos de casuística). La Sección 3 contiene la lista de requisitos específicos con prioridad y trazabilidad.

2 Descripción general

2.1 Perspectiva del producto

El sistema es un producto independiente (aplicación de consola) que podría integrarse en el futuro con un sistema mayor (p. ej. sitio web del hotel o sistema contable). Actualmente no requiere integración externa obligatoria; sin embargo, se diseña para permitir una futura API REST.

2.2 Funcionalidad del producto

- Crear, modificar, cancelar reservas con control de disponibilidad.
- Registrar, actualizar y consultar clientes (historial de reservas).
- Gestionar habitaciones: tipos, tarifas, estados (disponible, ocupada, mantenimiento).
- Procesamiento de pagos: validación de medios, confirmaciones, logging.
- Generación de reportes simples (listado de reservas activas, ocupación por fecha).
- Documentación técnica (clases, métodos, diagrama simple).

2.3 Características de los usuarios

Tipo de usuario	Recepcionista / Administrador del hotel
Formación	Conocimientos básicos de computación
Habilidades	Introducir y consultar datos
Actividades	Usará CLI para gestionar reservas

Tipo de usuario	Administrador técnico
Formación	DevOps
Habilidades	Desarrollo o administración de sistema
Actividades	Instalar y mantener el sistema

Tipo de usuario	Auditor
Formación	Contabilidad
Habilidades	Manejo básico de computación, Contabilidad.
Actividades	Revisar reportes de pagos y actividad.

2.4 Restricciones

- Lenguaje: Python 3.10+ (POO).
- Plataforma: PC con sistema operativo Windows/Linux (terminal).
- Tiempo de entrega: máximo 15 días calendario.
- Interfaz: solo consola (CLI) en esta versión.
- Base de datos local (SQLite o archivos JSON).
- Método de trabajo: Scrum (3 sprints de 5 días).

2.5 Suposiciones y dependencias

- Se asume disponibilidad del entorno Python en el servidor/PC destino.
- Se asume que no habrá integración obligatoria con pasarelas reales de pago en la primera entrega (pagos simulados o modos de validación).

- Dependencia de librerías Python estándar y algunas externas (ej. sqlite3, pytest).

2.6 Evolución previsible del sistema

- Añadir API REST para integración con front-end web.
- Incorporación de interfaz web/GUI.
- Integración con pasarelas de pago reales.
- Multi-hotel / multi-sede.
- Internacionalización (M/I18n).

3 Requisitos específicos

Campo	Contenido
Número de requisito:	RF-01
Nombre de requisito:	Registro de Clientes
Tipo:	☒ Requisito ☐ Restricción
Fuente del requisito:	Documento de Alcance del Proyecto – Product Owner
Prioridad del requisito:	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/Opcional

Campo	Contenido
Número de requisito:	RF-02
Nombre de requisito:	Consulta de Clientes
Tipo:	☒ Requisito ☐ Restricción
Fuente del requisito:	Documento de Alcance / Product Owner
Prioridad del requisito:	☒ Media/Deseado ☐ Alta/Esencial ☐ Baja/Opcional

Número de requisito:	RF-03
Nombre de requisito:	Creación de Reservas
Tipo:	☒ Requisito ☐ Restricción
Fuente del requisito:	Documento de Alcance / Revisión Sprint 1
Prioridad del requisito:	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/Opcional

Número de requisito:	RF-04
Nombre de requisito:	Modificación y Cancelación de Reservas
Tipo:	☒ Requisito ☐ Restricción
Fuente del requisito:	Product Backlog / Cliente
Prioridad del requisito:	☒ Alta/Esencial ☐ Media/Deseado ☐ Baja/Opcional

Número de requisito:	RF-05
Nombre de requisito:	Gestión de Habitaciones

<i>Tipo:</i>	☒ Requisito ☐ Restricción
<i>Fuente del requisito:</i>	Documento de Alcance
<i>Prioridad del requisito:</i>	☒ Alta/Eencial ☐ Media/Deseado ☐ Baja/Opcional

<i>Número de requisito:</i>	RF-06
<i>Nombre de requisito:</i>	Procesamiento de Pagos
<i>Tipo:</i>	☒ Requisito ☐ Restricción
<i>Fuente del requisito:</i>	Sprint 2 / Cliente
<i>Prioridad del requisito:</i>	☒ Alta/Eencial ☐ Media/Deseado ☐ Baja/Opcional

<i>Número de requisito:</i>	RF-07
<i>Nombre de requisito:</i>	Generación de Reportes
<i>Tipo:</i>	☒ Requisito ☐ Restricción
<i>Fuente del requisito:</i>	Reunión con Cliente / Sprint 3
<i>Prioridad del requisito:</i>	☒ Media/Deseado ☐ Alta/Eencial ☐ Baja/Opcional

3.1 Requisitos comunes de los interfaces

Esta sección describe de forma detallada las **entradas y salidas del sistema de software**, así como los tipos de interfaces necesarias para la interacción con el usuario, el hardware, otros sistemas y posibles medios de comunicación.

- **IU-1 (CLI - Menú principal):** El sistema mostrará un menú principal con opciones numeradas: 1) Gestión Reservas, 2) Gestión Clientes, 3) Gestión Habitaciones, 4) Pagos, 5) Reportes, 6) Salir.
 - **Entrada:** selección numérica.
 - **Salida:** ejecución del módulo solicitado.
 - **Prioridad:** Alta.
- **IU-2 (Formularios CLI):** Para creación/edición se solicitarán campos en línea (ej.: nombre cliente, documento, fecha entrada/salida, tipo habitación, método de pago).
 - Validaciones en cada campo (no vacíos, formatos de fecha, tarifas numéricas).

3.1.1 Interfaces de hardware

- **HW-1:** Sistema compatible con PC estándar (CPU x86/x64, 2GB RAM mínimo).
- **HW-2:** Almacenamiento: mínimo 100 MB libre para datos y logs.

3.1.2 Interfaces de software

- **SW-1:** Requiere Python 3.10+ instalado.
- **SW-2:** Base de datos: SQLite (módulo sqlite3) o ficheros JSON (opcional).

- **SW-3:** Herramientas de pruebas: pytest.

3.1.3 Interfaces de comunicación

- **COM-1** (NA en primera versión): No hay integración externa obligatoria. Se deberá documentar un posible endpoint REST para futuras integraciones (apéndice).

3.2 Requisitos funcionales

Los requisitos funcionales definen **las** acciones fundamentales que debe realizar el software del sistema de gestión de reservas para un hotel, desde la recepción y validación de entradas hasta la generación de salidas y respuestas ante errores o excepciones.

3.2.1 RF-1 Registro de Clientes

Descripción general:

El sistema debe permitir registrar nuevos clientes ingresando los siguientes datos: nombre, apellidos, tipo y número de documento, correo electrónico, teléfono y dirección.

El número de documento será único para cada cliente.

El sistema validará los campos obligatorios y mostrará un mensaje de confirmación cuando el registro sea exitoso.

Entradas: Datos personales del cliente.

Salidas: Mensaje de confirmación y código único de cliente (ID).

Errores posibles: Campos vacíos, duplicidad de documento.

Postcondición: Cliente almacenado correctamente en la base de datos.

3.2.2 RF-2 Consulta de Clientes

Descripción del requisito:

El sistema permitirá consultar los datos de los clientes registrados, ya sea por nombre, documento o ID.

Debe mostrar la información completa y el historial de reservas.

Entradas: Criterio de búsqueda (nombre, documento o ID).

Validaciones: Campo no vacío.

Secuencia de operaciones:

1. Recibir criterio de búsqueda.
2. Consultar base de datos.
3. Mostrar resultado en pantalla.

Salidas: Datos del cliente y sus reservas.

Errores: Cliente no encontrado.

Relación entrada/salida: Criterio de búsqueda → datos del cliente.

Postcondición: No se modifican datos.

3.2.3 RF-3 Creación de Reservas

Descripción del requisito:

El sistema permitirá crear reservas para clientes, verificando la disponibilidad de habitaciones en las fechas solicitadas y calculando el costo total según el tipo de habitación.

Entradas:

ID del cliente, tipo de habitación, fecha de entrada, fecha de salida, método de pago.

Validaciones:

El cliente debe existir, las fechas deben ser válidas (fecha de salida posterior a la de entrada) y la habitación debe estar disponible.

Secuencia de operaciones:

1. Solicitar los datos de reserva.
2. Validar cliente y fechas.
3. Consultar disponibilidad de habitación.
4. Calcular el valor total.
5. Registrar la reserva y bloquear la habitación.

Salidas:

ID de reserva y mensaje de confirmación.

Errores:

Fechas inválidas, habitación ocupada o cliente inexistente.

Relación entrada/salida:

Fechas válidas y cliente existente → reserva generada.

Postcondición:

La reserva queda almacenada en la base de datos y la habitación se marca como ocupada.

3.2.4 RF-4: Modificación y Cancelación de Reservas

Descripción del requisito:

El sistema permitirá modificar la información de una reserva existente (como fechas o tipo de habitación) y también cancelar reservas. Toda acción debe ser validada antes de aplicarse.

Entradas:

ID de la reserva, nuevos datos de reserva o motivo de cancelación.

Validaciones:

Verificar existencia de la reserva, disponibilidad de nuevas fechas o habitaciones y cumplimiento de políticas de cancelación.

Secuencia de operaciones:

1. Ingresar ID de reserva.
2. Validar la existencia de la reserva.
3. Aplicar modificación o cancelación.
4. Actualizar la base de datos.

Salidas:

Mensaje de confirmación o error.

Errores:

Reserva inexistente, fechas en conflicto o habitación no disponible.

Relación entrada/salida:

Datos de reserva válidos → reserva actualizada o cancelada.

Postcondición:

La reserva queda actualizada o eliminada del sistema según la acción ejecutada.

3.2.5 RF-5: Gestión de Habitaciones

Descripción del requisito:

El sistema permitirá registrar, modificar y eliminar habitaciones. Cada habitación incluirá información como tipo, tarifa y estado (disponible, ocupada, mantenimiento).

Entradas:

Código de habitación, tipo, tarifa, estado.

Validaciones:

Código único, tipo válido, tarifa numérica positiva.

Secuencia de operaciones:

1. *Solicitar información de la habitación.*
2. *Validar los datos ingresados.*
3. *Registrar o actualizar el registro.*

Salidas:

Mensaje de confirmación del registro o actualización.

Errores:

Código duplicado, campos vacíos o datos inválidos.

Relación entrada/salida:

Datos válidos → habitación registrada o actualizada.

Postcondición:

Habitación almacenada correctamente en la base de datos.

3.2.6 RF-6: Procesamiento de Pagos

Descripción del requisito:

El sistema procesará los pagos de reservas, validando el monto total y el método de pago antes de confirmar la transacción.

Entradas:

ID de reserva, monto a pagar, método de pago.

Validaciones:

Verificar existencia de la reserva, monto correcto y método de pago permitido.

Secuencia de operaciones:

1. *Solicitar datos del pago.*

2. Validar información.
3. Registrar el pago.
4. Cambiar el estado de la reserva a "Pagada".

Salidas:

Mensaje de confirmación y comprobante del pago.

Errores:

Reserva inexistente o monto incorrecto.

Relación entrada/salida:

Datos válidos de pago → reserva confirmada.

Postcondición:

El pago queda registrado y la reserva actualizada como pagada.

3.2.7 RF-6: Generación de Reportes

Descripción del requisito:

El sistema permitirá generar reportes sobre ocupación, reservas y transacciones. Los reportes podrán visualizarse en consola o exportarse a formato CSV.

Entradas:

Fecha de inicio, fecha de fin, tipo de reporte.

Validaciones:

Las fechas deben ser válidas y el rango no debe estar vacío.

Secuencia de operaciones:

1. Solicitar parámetros de consulta.
2. Validar fechas y tipo de reporte.
3. Consultar base de datos.
4. Mostrar o exportar resultados.

Salidas:

Reporte generado en pantalla o archivo CSV.

Errores:

Fechas inválidas o ausencia de datos.

Relación entrada/salida:

Rango de fechas → reporte correspondiente.

Postcondición:

Reporte generado y disponible para descarga o visualización.

3.3 Requisitos no funcionales

3.3.1 Requisitos de rendimiento

El sistema debe mantener un nivel de rendimiento adecuado para garantizar una experiencia fluida y eficiente durante la gestión de reservas, incluso con una carga moderada de trabajo.

Requisitos específicos:

1. El 95 % de las operaciones de creación, consulta o actualización (CRUD) sobre clientes, habitaciones y reservas deben completarse en menos de 1 segundo en un entorno con 4 GB de RAM y procesador de un solo núcleo.
2. Las consultas de disponibilidad de habitaciones en rangos de fechas de hasta 365 días deben ejecutarse en menos de 2 segundos.
3. El sistema debe ser capaz de procesar hasta 50 transacciones de pago simuladas por minuto sin pérdida de datos.
4. Los reportes generados (ocupación o reservas) deben completarse en menos de 3 segundos para bases de datos con hasta 10 000 registros.
5. El sistema debe permitir al menos 3 usuarios concurrentes locales (por turnos o terminales) sin interferencias en los datos.

3.3.2 Seguridad

El sistema debe proteger la información sensible del hotel y de los clientes frente a accesos no autorizados, errores de manipulación o pérdida accidental.

Medidas y requisitos:

1. Autenticación básica mediante credenciales para el usuario administrador del sistema.
2. Validación y control de acceso a operaciones críticas (por ejemplo, eliminación de registros o modificación de tarifas).
3. Implementación de logs de auditoría que registren usuario, acción, fecha y descripción del evento.
4. Almacenamiento seguro de la base de datos con permisos restringidos a nivel de sistema operativo (sólo lectura/escritura para el usuario del sistema).
5. En caso de integrarse pagos reales en versiones futuras, se emplearán técnicas de tokenización y cifrado AES para proteger los datos de tarjetas.
6. Comprobaciones de integridad de datos mediante verificación de hashes en los archivos de respaldo.
7. Desactivación del acceso remoto no autenticado a los archivos del sistema.

3.3.3 Fiabilidad

El sistema debe operar de forma confiable, minimizando fallos y asegurando la recuperación ante incidentes menores.

Requisitos específicos:

1. El sistema debe alcanzar un tiempo medio entre fallos (MTBF) superior a 30 días bajo uso normal.
2. En caso de error o cierre inesperado, el sistema debe preservar los datos registrados hasta el último guardado.
3. Los fallos críticos deben registrarse automáticamente en un archivo de logs para su análisis.

4. El proceso de respaldo manual o automático debe permitir recuperar el sistema en menos de 1 hora en caso de corrupción de datos.

3.3.4 Disponibilidad

El sistema debe estar disponible para los usuarios del hotel durante las horas operativas, con tiempos de inactividad mínimos.

Requisitos específicos:

1. Disponibilidad mínima esperada del 99 % durante las horas de operación (8 a.m. – 10 p.m.).
2. Las actualizaciones o mantenimientos programados deberán realizarse fuera del horario operativo.
3. En caso de interrupción, la aplicación debe mostrar un mensaje de error descriptivo y guardar la información en memoria antes del cierre.

3.3.5 Mantenibilidad

El sistema debe ser fácilmente mantenible y actualizable por el equipo de desarrollo o personal técnico asignado.

Requisitos específicos:

1. El código fuente debe estar documentado conforme a las guías PEP8 y contener comentarios en las funciones principales.
2. Se deberán desarrollar pruebas unitarias que cubran al menos el 60 % de las funcionalidades críticas (reservas, pagos y clientes).
3. El sistema deberá contar con un archivo README que detalle la instalación, configuración y ejecución del software.
4. Las tareas de mantenimiento (limpieza de logs, backups, actualizaciones) deberán realizarse semanalmente por el administrador técnico.
5. La estructura modular del código debe permitir sustituir o mejorar componentes (por ejemplo, el motor de base de datos) sin afectar la lógica del negocio.

3.3.6 Portabilidad

El software debe ser portable y fácilmente ejecutable en diferentes plataformas, sin necesidad de configuraciones complejas.

Requisitos específicos:

1. El sistema será 100 % compatible con Python 3.10 o superior.
2. Deberá funcionar indistintamente en sistemas operativos Windows 10, Windows 11 y Ubuntu 20.04+.
3. La base de datos se implementará en SQLite, una tecnología embebida, ligera y multiplataforma.
4. No se emplearán dependencias externas que restrinjan la ejecución del sistema en otros entornos.
5. El porcentaje de código dependiente de la plataforma no deberá superar el 5 % del total.

6. Las rutas y configuraciones se establecerán de forma relativa, evitando dependencias absolutas del sistema operativo.

3.4 Otros requisitos

3.4.1 Requisitos legales y normativos

1. El sistema debe cumplir con la Ley de Protección de Datos Personales (Ley 1581 de 2012 – Colombia), garantizando el consentimiento informado del cliente para el tratamiento de datos.
2. No se almacenarán datos de tarjetas de crédito reales, salvo mediante tokenización en futuras integraciones.
3. Los datos personales deben poder eliminarse si el cliente solicita la supresión conforme a la política de privacidad.

3.4.2 Requisitos culturales y lingüísticos

1. El sistema operará en **idioma** español (es_CO) por defecto.
2. Los mensajes de la interfaz deben ser claros, formales y comprensibles para personal administrativo con nivel educativo medio.
3. Los reportes y archivos exportados deberán generarse en formato **UTF-8** para soportar caracteres latinos.

3.4.3 Requisitos operativos

1. El sistema debe generar copias de respaldo diarias en formato CSV o SQL.
2. Los respaldos podrán almacenarse de manera local o en la nube (por ejemplo, Google Drive o OneDrive).
3. El sistema debe poder instalarse sin conexión a Internet.
4. Se recomienda ejecutar auditorías mensuales de seguridad y mantenimiento preventivo.

4 Apéndices

4.1.1 Apéndice A – Modelo de datos

Entidades principales del sistema y sus relaciones:

- **Cliente:** cliente_id, nombre, apellidos, documento, contacto, estado.
- **Habitación:** habitacion_id, tipo, tarifa, estado.
- **Reserva:** reserva_id, cliente_id, habitacion_id, fechas, total, estado.
- **Pago:** pago_id, reserva_id, monto, método, estado.
- **LogEvento:** log_id, usuario, acción, descripción, fecha.

4.1.2 Apéndice B – Escenarios de prueba

1. **Registro de cliente** → validar duplicidad y formato de correo.
2. **Creación de reserva** → verificar disponibilidad y cálculo correcto de tarifa.
3. **Pago confirmado** → actualizar reserva y generar recibo.
4. **Cancelación de reserva** → liberar habitación y registrar log.
5. **Error de conexión a BD** → registrar incidente sin pérdida de datos.

4.1.3 Apéndice C – Evolución futura

1. Desarrollo de una interfaz gráfica o web.
2. Integración con pasarelas de pago reales (PayU, Stripe).
3. Soporte multi-hotel y multiusuario.
4. Implementación de notificaciones por correo al cliente.

4.1.4 Apéndice D – Documentación técnica

- Manual de instalación y configuración (README).
- Manual de usuario final.
- Diagrama de clases UML.
- Plan de pruebas unitarias.
- Scripts de mantenimiento y respaldo.